

Appello TAP del 02/02/2026

Avete a disposizione **2 ore (2 ore e 30 minuti per chi ha DSA)**. La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** (6+5 = 11 punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 su ognuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare “cose”, la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa NON fare).

Per la consegna, caricate su AulaWeb la **solution “zippata”**, ovvero “zippate” la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il **SanityCheck** (presente anche in versione `.txt` su AulaWeb) non verrà considerato. Dopo aver caricato la solution, per paranoia, scaricate il vostro zip, scompattatelo da un'altra parte e verificate che compili tutto.

Esercizio 1 (11 punti)

In un progetto *class library* implementare l'extension method `MyTake<T>` che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un intero `count`

in base a `count` dovrà restituire una sequenza:

- con *tutti* gli elementi della sequenza `source`, quando `count == -1`
- vuota, quando `count == 0`
- con i primi `count` elementi di `source` (o tutti quelli che ci sono, se la sequenza termina prima di `count` elementi), quando `count > 0`

Inoltre, il metodo dovrà **immediatamente** (=senza richiedere nessuna enumerazione) lanciare `ArgumentOutOfRangeException` se `count < -1`.

Ricordate che gli extension method devono essere **static** dentro una classe **static**.

Vincoli

- È vietato chiamare il metodo standard `Take`, o qualunque altro metodo standard equivalente, che “risolva” l'esercizio al posto vostro
- Il metodo non deve enumerare l'intera sequenza se non necessario
- L'eccezione per `count < -1` deve essere lanciata **immediatamente** senza aspettare che qualcuno cerchi di enumerare la sequenza

Sanity check

Lo scopo di questo test è solo verificare che la *signature* del vostro extension method sia abbastanza simile a quanto atteso da permettere di compilare ed eseguire i test.

```
[Fact]
public void SanityCheck() {
    _ = Array.Empty<int>().MyTake(123);
    _ = new [] { "ciao" }.MyTake(-1);
    _ = new List<double> { 3.14, 1.23 }.MyTake(10000);
}
```

Esercizio 2 (6 punti)

Implementare, usando xUnit ed eventualmente Moq, i seguenti test relativi all'esercizio 1 che verifichino che con `count` uguale a

1. -1 vengano restituiti **tutti** gli elementi di una sequenza non vuota
2. -1 venga restituita una sequenza vuota a partire dalla sequenza vuota
3. 0 venga restituita una sequenza **vuota** (con `source` non vuota)
4. 3 restituisca i primi tre elementi da una lista di 5 stringhe
5. 3 restituisca i primi due elementi da un array di 2 double
6. -2 lanci `ArgumentOutOfRangeException` immediatamente (cioè senza enumerare la sequenza)

Esercizio 3 (5 punti)

In questo esercizio andiamo a testare una classe diversa, `MyPrinter`, che dipende dalla seguente interfaccia `ITextOutput` (simile a quella vista a lezione). Riportiamo per comodità il codice di entrambe, ma le troverete già implementata nel `.txt` che contiene il sanity-check.

```
public interface ITextOutput {
    void Write(string s);
}

public class MyPrinter {
    private readonly ITextOutput _output;

    public MyPrinter(ITextOutput output) => _output = output;

    public void PrintFirstN<T>(IEnumerable<T> source, int count) {
        foreach(var el in source.Take(count))
            _output.Write(el?.ToString() ?? "boh");
    }
}
```

Scrivete uno o più test, opzionalmente usando Moq, che verifichino che il metodo `MyPrinter.PrintFirstN` chiami il metodo `Write` dell'interfaccia `ITextOutput` (il numero corretto di volte/con gli argomenti corretti).

Nota: il test dovrà avere successo con questo codice, ma dovrà fallire se andate a commentare/cancellare il corpo del metodo `PrintFirstN`.